The main idea is to parse the document and detect its type. Then, it loads documents (PDF or DOCX) using the LangChain.js community packages, specifically `PDFLoader` and `DocxLoader`. Those are convenient functions that parse a document and return a list of `Document` types, which we can further use for splitting it up into chunks for summarization.

Listing 7.1 Loading different document types

```
import { PDFLoader } from "@langchain/community/document_loaders/fs/pdf";
import { DocxLoader } from "@langchain/community/document_loaders/fs/docx";
import path from "path";

const loaders = {
  pdf: PDFLoader,
  docx: DocxLoader,
};

const loadDocumentFromFile = async (filePath) => {
  const fileExtension = path.extname(filePath)
.substring(1).toLowerCase();
  const LoaderClass = loaders[fileExtension];

  if (!LoaderClass) {
    console.error(`No loader found for file extension: ${fileExtension}`);
    return null;
  }

  const loader = new LoaderClass(filePath);
  return await loader.load();
};

loadDocumentFromFile("./data/aesops-fables.pdf")
  .then((documents) => {
    console.log(`Loaded ${documents.length} documents.`);
  })
  .catch((error) => {
    console.error("Error loading documents:", error);
  });
```

Imports the PDFLoader class from the LangChain community package to handle PDF documents

Imports the DocxLoader class for handling DOCX files

Functions to load a document from a specified file path

Calls the loadDocumentFromFile function with a specific PDF file path

Once we have the parsed documents, we need to split them into chunks that will be used for summarization and for fitting the context window that many LLMs impose limits on.

> **NOTE** The `PDFLoader` from LangChain.js serves as a reasonable middle ground for parsing PDF documents for LLMs, but it's not perfect. The analysis of PDF content using these advanced models remains an ongoing area of study aiming to improve reliability.

SPLITTING THE DOCUMENT INTO MANAGEABLE CHUNKS

Before processing large documents, it's important to split them into smaller, manageable chunks. This ensures that the language model can handle the content efficiently without losing context. LangChain provides two common approaches: